ARCHITECTURE OVERVIEW

Mamba: Selective State-Space Models

Efficiency
O(n) Linear Scale

Innovation
Selectivity Scan

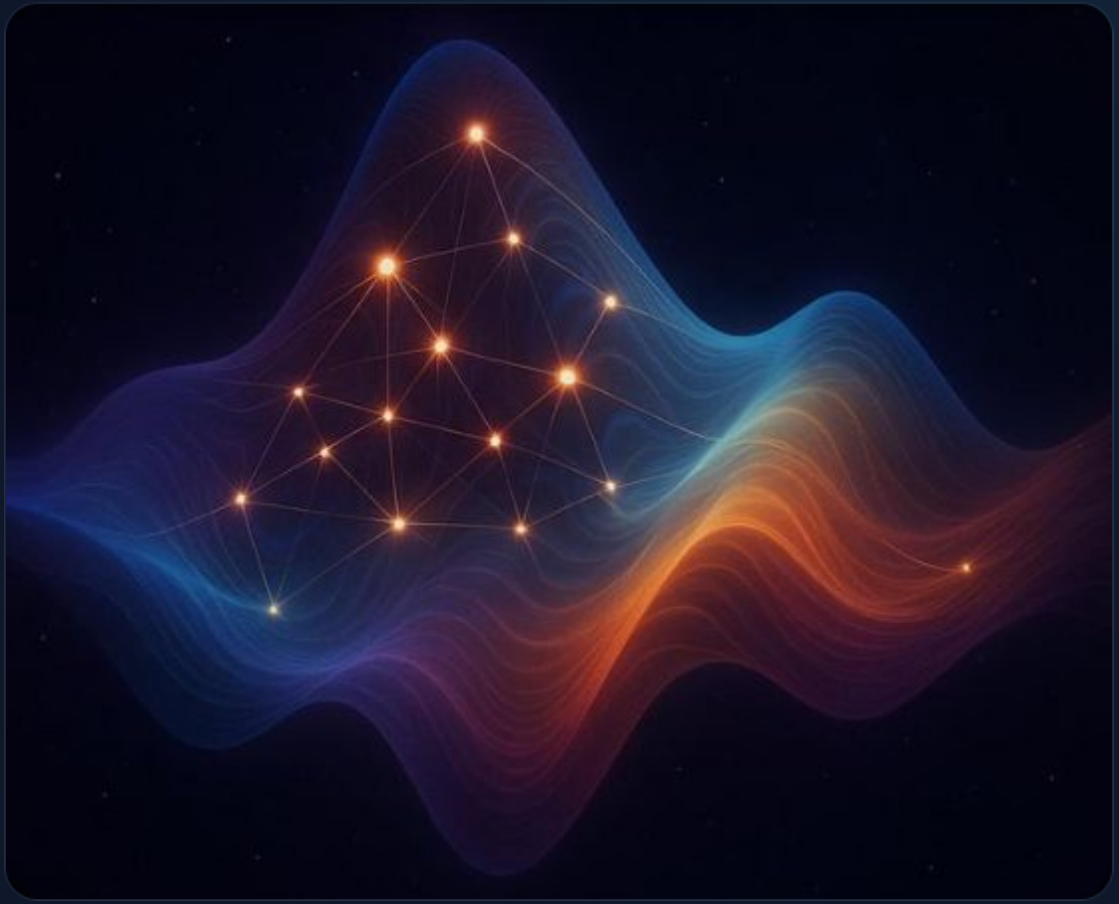
From Mathematical Foundations to Linear-Time Implementation

The Transformer Bottleneck

Self-Attention Scalability

Transformers have revolutionized AI, but suffer from **quadratic complexity** relative to sequence length. Every token attends to every other token.

- **Inference Bottleneck:** High KV-cache memory costs.
- **Memory:** $O(n^2)$ complexity limits long-context usage.
- **Throughput:** Decelerates rapidly as sequence grows.



Continuous Time SSM Math

The LTI System

$$\dot{h}(t) = Ah(t) + Bx(t)$$

$$y(t) = Ch(t) + Dx(t)$$

LTI equations are common way to describe state space systems in modern control theory.

Key Parameters

The system is defined by four core matrices mapping inputs to internal state and outputs:

- **State h(t):** Compressed history summary.
- **Transition A:** Governs state evolution.
- **Input B:** How current input affects state.

Discretization: Zero-Order Hold

Neural networks operate on discrete tokens. We must convert the continuous matrices into discrete counterparts using a time-step Δ.

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = (A\Delta)^{-1}(e^{A\Delta} - I)B$$

Discrete Recurrence

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k$$

Operates like an RNN for efficient serial inference.

Global Convolution

$$y = x * \bar{K}$$

Enables Transformer-like parallelization for training.

The Selective Scan Innovation

Content-Aware Selectivity

Instead of fixed weights, Mamba makes discretization parameters input-dependent:

$$\Delta_k = f_\Delta(x_k), \quad B_k = f_B(x_k)$$

- Selective retention of relevant tokens.
- Intentional forgetting of noise tokens.
- Parallel implementation via Associative Scan.

Diagonal A Complexity Benefits

State Evolution Cost

Restricting A to diagonal form allows element-wise operations, reducing discretization and state update complexity by an order of magnitude.

MambaBlock Design

Dual-Path Processing

- **X-Branch:** Contextual Conv + SSM.
- **Z-Branch:** Multiplicative SiLU gating.
- **Fused IO:** GPU-aware kernel fusion.

The input undergoes parallel processing across a gated linear unit and the selective SSM path.

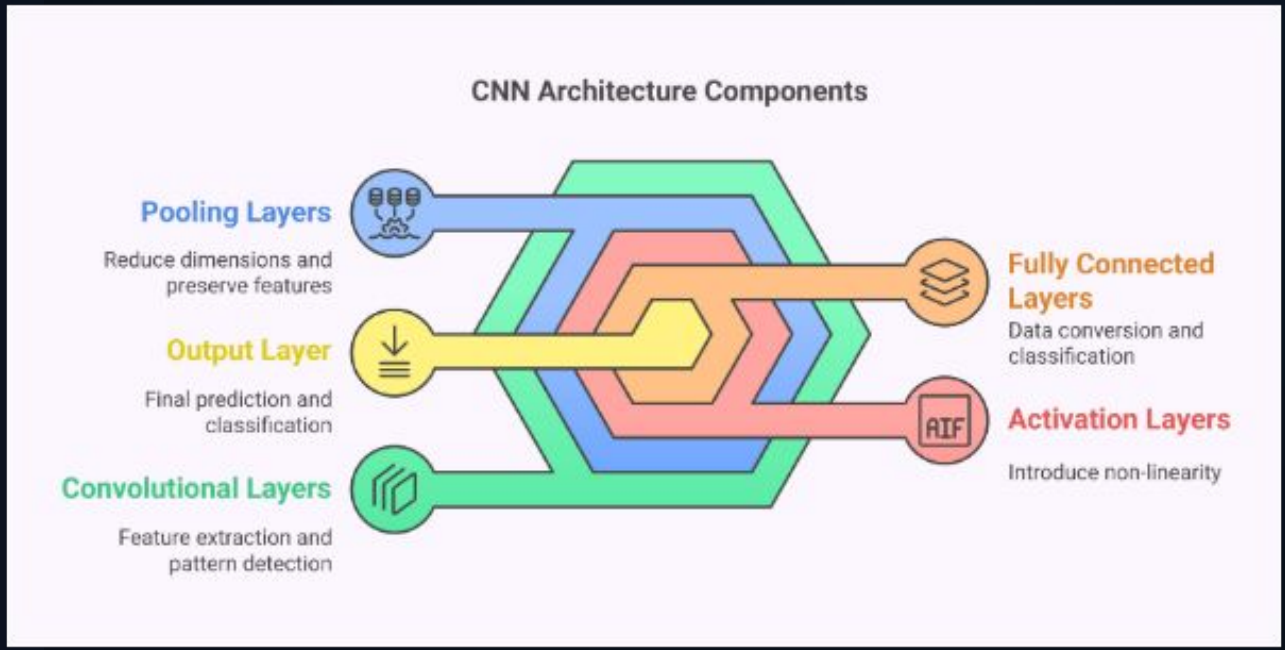


Diagram on right illustrates the tensor flow across projection layers and kernels.

Design Choice: Log-Space A

Parameterization

Storing A in log-space ensures eigenvalues remain negative after exponentiation.

$$A = -\exp(A_{log})$$

Initialization

Geometric timescale spread captures patterns across different temporal resolutions.

$$A_n = -(n+1)$$

Design Choice: Delta Bottleneck

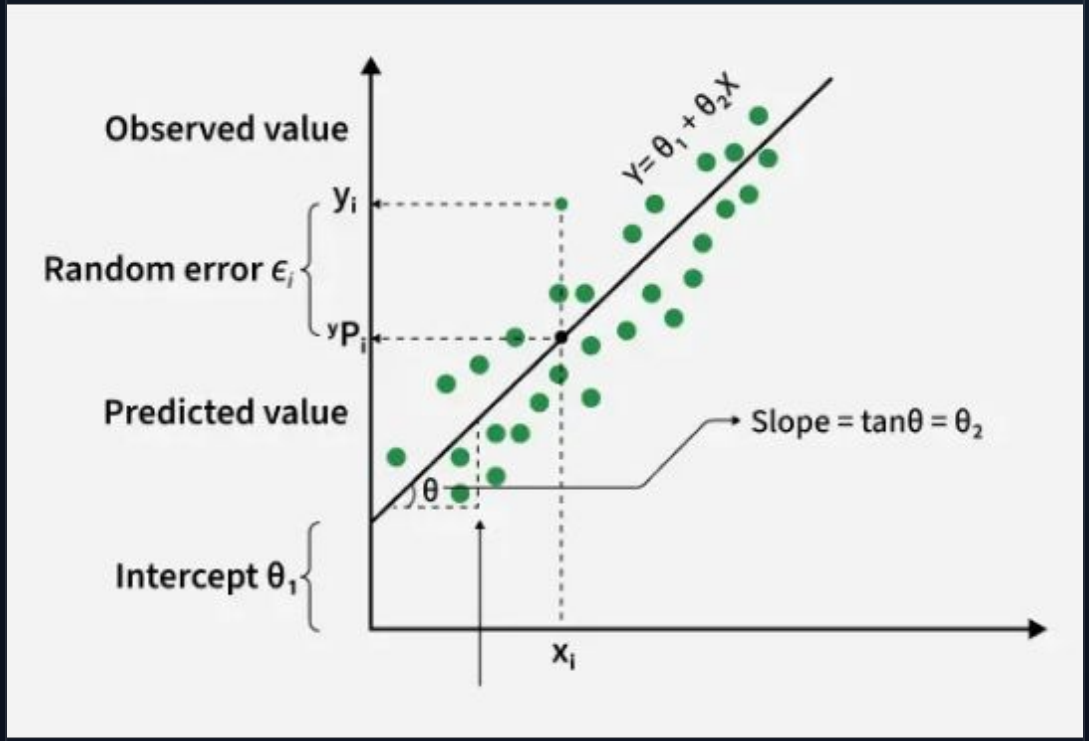
Low-Rank Delta Projection

Δ

$$\Delta = \text{clamp}\left(\text{softplus}\left(W_{up} W_{down} x + b\right)\right)$$

- Dimension reduction to rank r
- Softplus ensures positivity.
- Clamping prevents numerical instability.

To reduce parameter count while maintaining expressivity, is projected via a bottleneck.



Design Choice: Depthwise Conv

Local Structure Prior

Mamba applies a 1D convolution (kernel=4) to ensure the model captures immediate neighbors.

- **Causal Padding:** ZERO future leakage.
- **Channel Isolation:** No cross-channel mixing.
- **Efficiency:** Lightweight pre-scan op.

The SiLU Multiplicative Gate

SiLU

ACTIVATION

Non-Linear State Filtering

The gate allows the model to learn content-dependent filtering of the hidden state readout.

$$y = SSM_{out} \odot SiLU(z_{branch})$$

Smooth and non-monotone gradient flow, outperforming ReLU in deep recurrent stacks.

Stacking the Model Stack

1. **Embeddings:** Tied weight projection.
2. **Residual Blocks:** Wrapped Mamba Layers.
3. **Pre-Norm:** RMSNorm for stability.
4. **Output Head:** Logit generation.

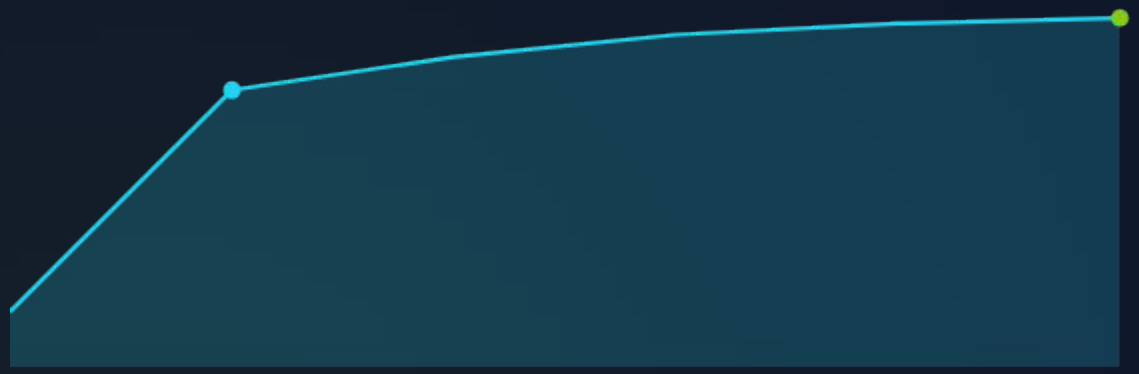
REPEATED L-TIMES FOR DEEP REPRESENTATION LEARNING

Training Improvements

Feature	Standard Regime	Mamba Implementation
Loss Function	Cross-Entropy	Label Smoothing $\varepsilon = 0.1$
Optimizer	Plain AdamW	AdamW + Delta-Weight Decay
Memory	Short Context	Enhanced Horizon via Regularization
Stability	Gradient Spikes	Smoothed Gradient Flow

Delta weight decay excludes biases to maintain the initial negative log-space distribution.

Results: Performance Gains



17.6%

Validation Loss Reduction

Optimization of delta decay and label smoothing provides significant convergence benefits with zero parameter overhead.

Steps: 500 Steps Training

Dataset: Shakespeare Char-LM

Testing & Validation

- **Discretization Check:** Verifying ZOH shape preservation.
- **Tensor Shapes:** Correct dimensions through SSM unrolling.
- **Causal Invariance:** No future leakage in padded convs.
- **Loss Bounds:** Ensuring smoothed CE stays bounded.
- **Optimizer Groups:** Correct param mapping for decay.
- **Generation:** Validating autoregressive text output.

Result: All 10 targeted unit tests passed successfully.

Optimal hardware

Mamba maximizes GPU efficiency

Parallel scan algorithm

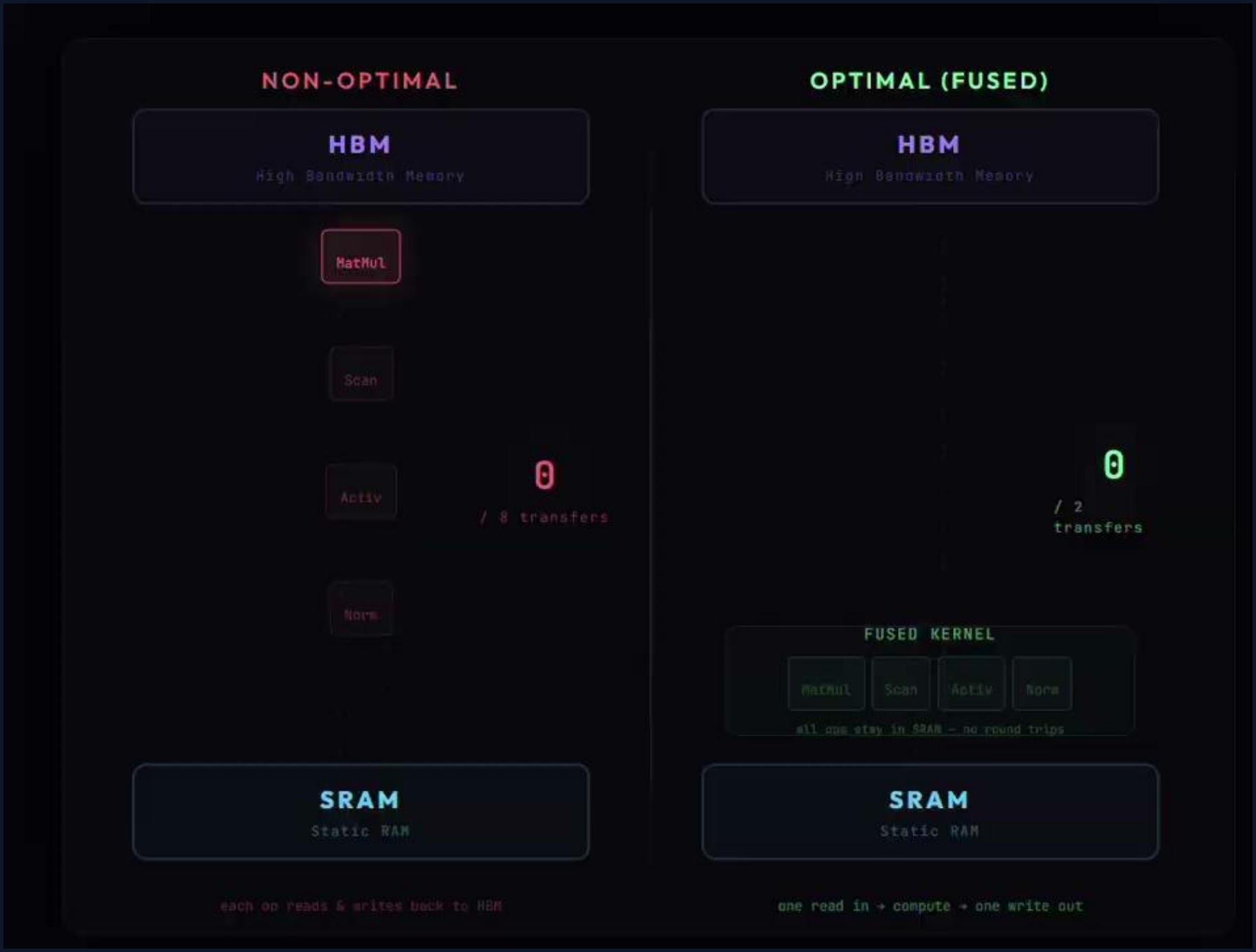
- ML models are trained within a sequential loop during training.
- Mamba uses a hardware-aware parallel scan to maximize GPU parallelism.
- This way, Mamba has a O(n) time complexity.

No attention matrix in memory

- Transformers store an $O(n^2)$ attention matrix that grows with sequence length.
- Mamba has fixed-size hidden states like RNN models.
- This way, even in long sequences, mamba is efficient in terms of memory.

Fused CUDA kernels

- Mamba was developed with low-level CUDA kernels that fuse operations together.
- This way, Mamba minimizes data transfer between GPU HBM (High Bandwidth Memory) and static RAM.
- Let's see an animation!


NON-OPTIMAL
HBM
High Bandwidth Memory
MatMul
Scan
Activ
Norm
0
/ 8 transfers
SRAM
Static RAM
each op reads & writes back to HBM
OPTIMAL (FUSED)
HBM
High Bandwidth Memory
0
/ 2
transfers
FUSED KERNEL
MatMul
Scan
Activ
Norm
all ops stay in SRAM — no round trips
SRAM
Static RAM
one read in → compute → one write out

Benchmarks

Sequential CIFAR-10

1. CIFAR-10: Dataset of 60,000 small (32×32) color images split across 10 everyday object categories.
2. Sequential Cifar-10 is just Cifar-10 but with flattened images pixel-by-pixel to act as a benchmark for an already existing sequence model.
3. Before being flattened, it is converted to grayscale so that each pixel only has 1 value.
4. This way, a 32×32 image is flattened into a 1D sequence of 1024 pixels as input, with the class label as the output.

- Input: Vector of 1024 values
- Target: Label of an image

- Training: ~45K samples
- Validation: ~5K samples
- Test: ~10K samples

- Thanks to the Long Range Arena benchmark, Sequential CIFAR-10 was widely popularized in 2020.

- ICLR 2021 - Long Range Arena: A Benchmark for Efficient Transformers https://arxiv.org/abs/2011.04006

Sequential CIFAR-10: Why we applied it

- RNNs struggle with long-range gradients. This way, sequential CIFAR-10 directly tests this weakness.
- Often transformers suffer from the $O(n^2)$ bottleneck. It tests how well models scale with sequence length.
- By also being used in other models, we can compare our results with other results from other papers.
- Show that, with preprocessing, the sequence models can classify images.

- ICLR 2021 - Long Range Arena: A Benchmark for Efficient Transformers: https://arxiv.org/abs/2011.04006

CIFAR-10: 5 epochs Normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.8530
Mamba	2	128	2	16	0.0005	—--
Fixed matrix Mamba	1	64	4	8	0.001	0.3783
Fixed matrix Mamba	2	128	2	16	0.0005	—-------

CIFAR-10: 10 epochs Normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.3702
Mamba	2	128	2	16	0.0005	—-------
Fixed matrix Mamba	1	64	4	8	0.001	0.5259
Fixed matrix Mamba	2	128	2	16	0.0005	—--------

CIFAR-10: 20 epochs Normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.0005	0.46
Mamba	2	128	2	16	0.0005	0.43
Fixed matrix Mamba	1	64	4	8	0.0005	0.34
Fixed matrix Mamba	2	128	2	16	0.0005	0.30

CIFAR-10: Time to run 20 epochs

- For the 20 epochs experiments it took nearly 165 minutes (2 hours and 45 minutes)
  - For normal mamba, 90 minutes (1h and 30 minutes)
  - For the Simpler Mamba, 75 minutes(1 hour and 15 minutes)

Listops:

1. ListOps was created to study the parsing ability of latent tree models.
2. For example, giving something like MAX(3, SUM(4, MIN(9, 2))) how could it give 6?
3. Popularized with Long Range Arena due to extending sequences to up to 2k tokens.

- Input: Huge sequences of nested mathematical operations
- Output: A number

- Training: ~96K samples
- Validation: ~2K samples
- Test: ~2K samples

NAACL 2018 - ListOps: A Diagnostic Dataset for Latent Tree Learning: https://arxiv.org/abs/1804.06028

Listops: Why we applied it

- Verifying the capability of processing long-range dependencies
- Understanding if the model can understand nested structures and apply operations in the correct order
- Also, if it can learn the tree structure needed to solve the task without being explicitly told how to parse.

NAACL 2018 - ListOps: A Diagnostic Dataset for Latent Tree Learning: https://arxiv.org/abs/1804.06028

Listops: 5 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	—---------
Mamba	2	128	2	16	0.0005	0.2805
Fixed matrix Mamba	1	64	4	8	0.001	—---------
Fixed matrix Mamba	2	128	2	16	0.0005	0.4065

Listops: 10 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	—-------
Mamba	2	128	2	16	0.0005	0.2995
Fixed matrix Mamba	1	64	4	8	0.001	—------
Fixed matrix Mamba	2	128	2	16	0.0005	0.5100

Listops: 20 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.49
Mamba	2	128	2	16	0.0005	0.30
Fixed matrix Mamba	1	64	4	8	0.001	0.51
Fixed matrix Mamba	2	128	2	16	0.0005	0.54

ListOps: 10 epochs vs 40 epochs

Epochs	Config	Test Acc
10	d=64, L=4	29.9%
40	d=64, L=4	59.9%

4x more training → 2x accuracy improvement

Listops: Time to run 20 epochs

- For the 20 epochs experiments it took nearly 560 minutes (9 hours and 20 minutes)
  - For normal mamba, 298 minutes(4h and 58 minutes)
  - For the Simpler Mamba, 262 minutes(4h and 22 minutes)

Selective Copying

1. Created in the mamba paper in 2024
2. The model sees a sequence of tokens where some are "relevant" and the rest are noise
3. The important tokens are scattered at unpredictable positions
4. The model must output only the important tokens in order
5. In the original paper the sequences were up to 110 tokens

- Input: [3] [0] [0] [7] [0] [0] [0] [1] [0] [5] [0] [0]...
- Output: 3 7 1 5…

- **Training:** 512 samples (seed=42)
- **Validation:** 128 samples (seed=99)
- **Test:** 128 samples (seed=7)

COLM 2024 - Mamba: Linear-Time Sequence Modeling with Selective State Spaces: https://arxiv.org/abs/2312.00752

Selective Copying: Why we applied it

- Selective copying isolates whether a model can decide what to keep and what to ignore based on content.
- While it is very simple, it is very hard for models without input-dependent dynamics.
- Easy to scale to see if the model memory holds up as sequences grow.
- Also, it shows whether your model solves the exact problem that selective SSMs were designed to address.
- While sCIFAR-10 and ListOps test long-range reasoning on structured data, Selective Copying specifically tests content-aware filtering.

COLM 2024 - Mamba: Linear-Time Sequence Modeling with Selective State Spaces: https://arxiv.org/abs/2312.00752

Selective Copying: 20 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.0677
Mamba	2	128	2	8	0.0005	0.0651
Fixed matrix Mamba	1	64	4	8	0.001	0.0885
Fixed matrix Mamba	2	128	2	8	0.0005	0.0898

Selective Copying: 40 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.3411
Mamba	2	128	2	8	0.0005	0.3255
Fixed matrix Mamba	1	64	4	8	0.001	0.3346
Fixed matrix Mamba	2	128	2	8	0.0005	0.2266

Selective Copying: 80 epochs normal Mamba vs. Mamba with fixed matrices

Experiment	Comb	d_model	n_layers	N	lr	Test Acc
Mamba	1	64	4	8	0.001	0.3385
Mamba	2	128	2	8	0.0005	0.3529
Fixed matrix Mamba	1	64	4	8	0.001	0.3776
Fixed matrix Mamba	2	128	2	8	0.0005	0.2513

Selective Copying: Time to run 80 epochs

- For the 80 epochs experiment it took nearly 255 minutes (4 hours and 15 minutes)
  - For normal mamba, 135 minutes(2h and 15 minutes)
  - For the Simpler Mamba, 120 minutes(2h)

Conclusion

- **GPU Efficiency:** Mamba relies on optimized GPU utilization to function effectively.
- **ListOps:** Fixed Matrix Mamba outperformed the standard version.
- **CIFAR:** Standard Mamba generally surpassed Fixed Matrix Mamba.
- **Selective Copying:** Fixed Matrix peaks higher with depth and training, while standard Mamba remains more consistent.
- **Tuning:** Results varied due to hyperparameter-driven over or underfitting.

THANK YOU

Questions?

Revolutionizing Sequence Modeling with Selective SSMs

Implementation
mamba-pytorch

Reference
Gu & Dao (2023)

GRU Baseline

Comparative Analysis with Mamba

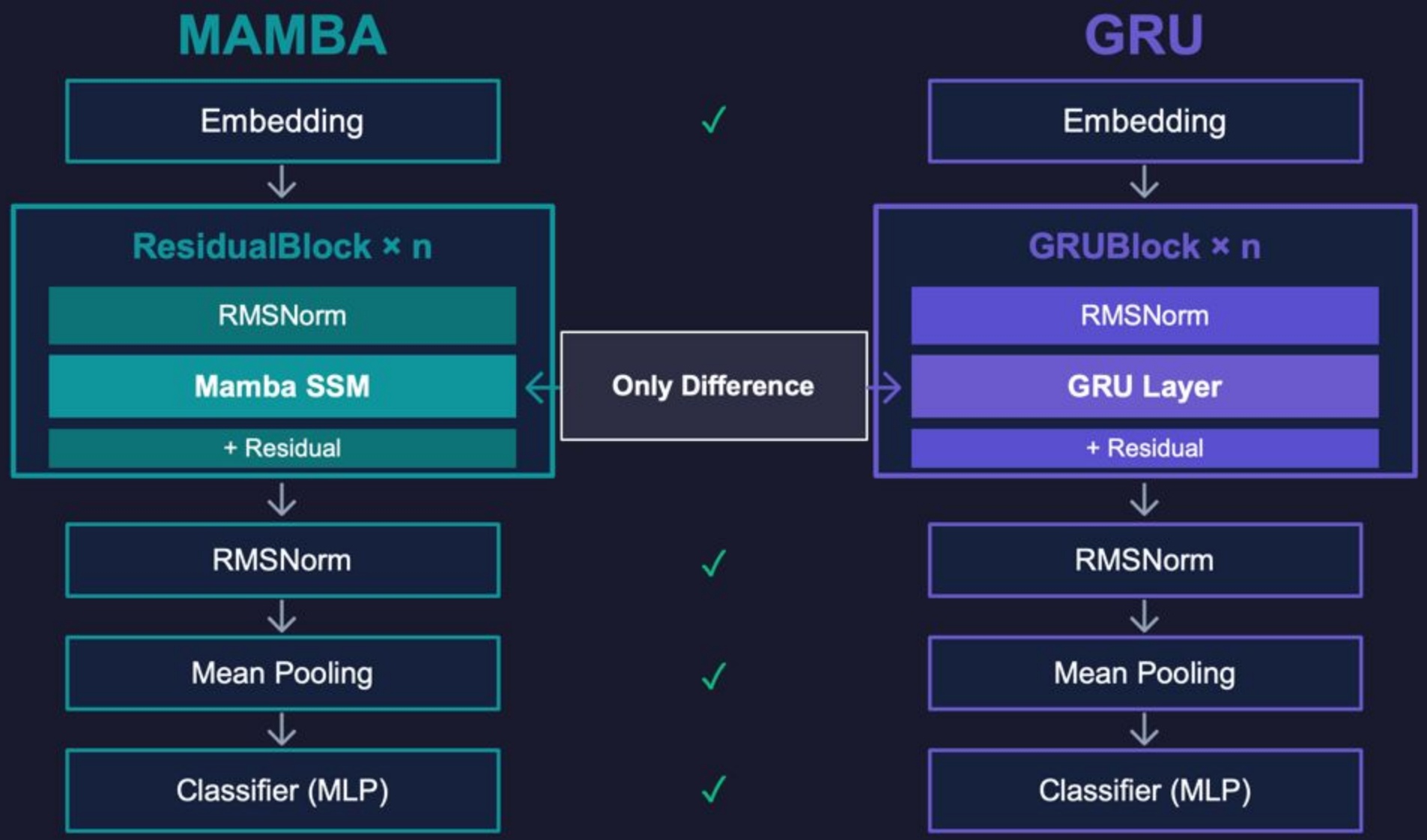
Why GRU?

- Established RNN baseline
- Gating mechanism comparison
- Same O(n) complexity as Mamba

Fair Comparison

- Same datasets & splits
- Same hyperparameters
- Same training setup

Architecture Comparison



Experimental Setup

Identical training conditions for fair comparison

Parameter	Combination 1	Combination 2
d_model	**64**	**128**
n_layers	**4**	**2**
Optimizer	AdamW	AdamW
Weight Decay	0.01	0.01

Dataset	Epochs	Seq Len
ListOps	**20**	2048
CIFAR-10	**20**	1024
Sel. Copying	**80**	128

Controlled Variables (Same for Both Models)

✓ Same datasets from HuggingFace (monteirot/lra-benchmarks)
✓ Same train/val/test splits ✓ Same batch sizes ✓ Same random seeds
✓ Same classifier head (2-layer MLP + GELU) ✓ Same pooling (mean) ✓ Same normalization (RMSNorm)

Results: GRU vs Mamba

ListOps (20 epochs)

Comb	Mamba	GRU	Δ
1	0.4900	0.2940	-0.1960
2	0.3000	0.5160	+0.2160

CIFAR-10 (20 epochs)

Comb	Mamba	GRU	Δ
1	0.46	0.5088	+0.0488
2	0.43	0.4656	+0.0356

Selective Copying (80 epochs)

Comb	Mamba	GRU	Δ
1	0.3385	0.4271	+0.0886
2	0.3529	0.4362	+0.0833

Key Findings

- **GRU remains a strong baseline** — simpler architecture competitive with or better than Mamba across all three tasks
- **Neither model dominates consistently** — results vary by task and configuration, suggesting task-architecture fit matters
- **More training may be needed** — 20 epochs for ListOps and CIFAR-10 may be insufficient for Mamba to fully converge
- **Controlled setup enables fair comparison** — identical everything except core layer isolates architectural differences

Conclusion

GRU vs Mamba: Summary

- GRU shows more stable and consistent performance across all three tasks
- Mamba achieves strong results in specific configurations (ListOps Comb 1) but is more sensitive to hyperparameter choice
- Neither model fully solves Selective Copying, suggesting both need longer training or longer sequences
- Controlled architecture design isolates core sequence modeling differences fairly

Mamba Advantages

- Strong on long sequences (ListOps, 2048 tokens)
- Theoretically more scalable
- Selective state space enables input-dependent filtering

GRU Advantages

- More stable across tasks and configs
- Faster convergence
- Simpler to tune and optimize